



Escala de Obreiros

ADVEC Vitória

Estrutura completa do projeto — para copiar, entender e dar continuidade

Repositório: jesimielbetel1-png/Tra-olife01

Branch: claude/weekly-schedule-system-008214

Pasta do sistema: escala/

Conteúdo: como copiar 100%, árvore de arquivos, o que cada um faz, fluxo de dados, modelo do banco e próximos passos.

1 Como levar tudo para o seu desktop

Os arquivos estão na branch `claude/weekly-schedule-system-008214`. Dois caminhos:

Caminho 1 — Baixar o ZIP (não precisa saber Git)

1. Acesse github.com/jesimielbetel1-png/Tra-olife01
2. No alto, troque a branch para `claude/weekly-schedule-system-008214`
3. Botão verde **Code** → **Download ZIP** e descompacte
4. A pasta que interessa é `escala/`

Caminho 2 — Clonar com Git (recomendado para desenvolver)

```
# baixa o projeto inteiro e entra na pasta da escala
git clone https://github.com/jesimielbetel1-png/Tra-olife01.git
cd Tra-olife01
git checkout claud/weekly-schedule-system-008214
cd escala
```

2 Árvore de arquivos (o que existe hoje)

Tra-olife01/	← repositório
├─ index.html	← app ANTIGO "Tração Life" (não é a escala)
├─ tracao_life_01.html	← cópia do app antigo
└─	
├─ escala/	← ★ O SISTEMA DE ESCALA
│ ├─ index.html	← página que carrega o app
│ ├─ styles.css	← aparência (tema ADVEC)
│ ├─ config.js	← SUAS CHAVES (Supabase/Telegram/WhatsApp)
│ ├─ data.js	← camada de dados (Supabase OU demonstração)
│ ├─ engine.js	← motor que monta a escala da semana
│ ├─ app.js	← todas as telas (login, obreiro, liderança)
│ └─ schema.sql	← banco + segurança (roda no Supabase)
├─ supabase/functions/	← o BOT do Telegram (3 funções)
│ └─ telegram-webhook/	← vincula o Telegram do obreiro
│ └─ notify-pendentes/	← cobra quem não lançou disponibilidade
│ └─ notify-escala/	← avisa a escala publicada
└─	
├─ README.md	← guia de instalação
├─ ESTRUTURA.md	← o mesmo conteúdo deste PDF
├─ netlify.toml	← publicar no Netlify
└─ vercel.json	← publicar no Vercel

3 O que cada arquivo faz

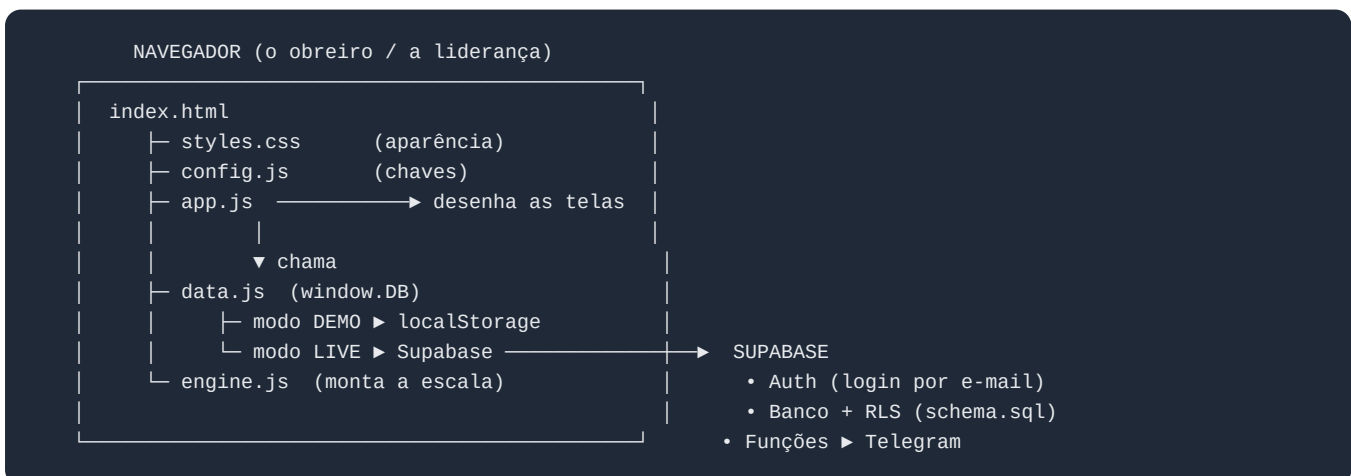
Frontend (o site que roda no navegador)

Arquivo	Papel	Você mexe?
<code>index.html</code>	Carrega os scripts na ordem certa e tem a <code><div id="app"></code> onde tudo é desenhado.	Raro
<code>styles.css</code>	Todo o visual: cores, botões, tabelas, cards, abas.	Aparência
<code>config.js</code>	Onde você cola a Project URL e a anon key do Supabase, o bot do Telegram e o WhatsApp. Vazio = modo demonstração.	SIM
<code>data.js</code>	O "meio de campo": uma única API (<code>window.DB</code>) com todas as operações. Decide sozinha entre Supabase (se há chaves) e demonstração (dados no navegador, com os 52 obreiros reais).	Novos campos
<code>engine.js</code>	O motor da escala : recebe obreiros, aptidões, disponibilidade e histórico e devolve quem vai em cada posição. Regras: balanceamento, Santa Ceia na 3ª terça e liderança (Jesimiel/Abner).	Regras
<code>app.js</code>	Todas as telas : login, tela do obreiro (disponibilidade + Telegram) e o painel da liderança (Escala, Disponibilidades, Obreiros, Frequência, Ajustes).	Telas/fluxo

Backend — Supabase (só quando você conecta de verdade)

Arquivo	Papel
<code>schema.sql</code>	Cria as tabelas, as regras de segurança (RLS) e o gatilho que faz toda conta nova nascer inativa . Roda uma vez no SQL Editor do Supabase.
<code>telegram-webhook</code>	Recebe as mensagens do bot; ao tocar em <i>Iniciar</i> , grava o chat do obreiro.
<code>notify-pendentes</code>	A liderança dispara para lembrar quem não lançou a disponibilidade da semana.
<code>notify-escala</code>	A liderança dispara depois de publicar; envia a cada obreiro as posições dele .

4 Como tudo se conecta (o fluxo dos dados)



Regra de ouro: `app.js` e `engine.js` **nunca** falam direto com o Supabase. Eles só chamam `window.DB` (em `data.js`). Por isso o mesmo código funciona na demonstração e no sistema de verdade — e você troca o motor de dados num lugar só.

5 Rodar no seu desktop (para testar)

O site é HTML puro — não precisa instalar nada para ver funcionando em modo demonstração. Sirva a pasta por HTTP:

Com Python (Mac/Linux já têm)

```
cd escala
python3 -m http.server 8000
# abra: http://localhost:8000
```

Com Node

```
cd escala
npx serve .
```

Com VS Code

Extensão **Live Server** → botão “Go Live”. É o editor recomendado para continuar.

Entre como **Ob. Jesimiel** ou **Ob. Abner** para ver o painel da liderança; entre como outro obreiro para ver a tela de disponibilidade.

6 Modelo de dados (as tabelas do banco)

Tabela	Guarda	Campos principais
obreiros	Uma linha por conta	id, email, nome, sexo, ativo, is_admin, prioridade, telegram_chat_id
aptidoes	Posições que cada um pode assumir (confidencial)	obreiro_id, posicao
disponibilidades	O que o obreiro lançou por semana	obreiro_id, semana, dom_manha, dom_noite, terca, quinta
escalas	Uma por semana	id, semana, status (rascunho/publicada)
escala_itens	Cada posição atribuída	escala_id, culto, data, posicao, obreiro_id
presencas	Frequência	item_id, presente, marcado_por
matriz_quantidade	Quantos por posição em cada culto	posicao, categoria, qtd_padrao

7 Para dar continuidade — próximos passos

1. **Abrir no VS Code** e rodar em modo demonstração (seção 5) para se familiarizar.
2. **Criar o projeto Supabase** e rodar o `schema.sql` (ver `README.md`).
3. **Preencher o `config.js`** com as chaves → o site vira “de verdade”.
4. **Virar admin** com o comando SQL do `README.md`.
5. **Publicar** no Vercel ou Netlify (configs já prontas na pasta).
6. **Ligar o Telegram** (BotFather + deploy das 3 funções).

Ideias de evolução (quando quiser)

- Importar o cadastro direto da planilha `.xlsx` para dentro do Supabase.
- Disparo **automático** do lembrete toda sexta (hoje é manual, por botão) — via agendamento (`pg_cron`) no Supabase.
- Relatório mensal de frequência em PDF.
- App instalável no celular (PWA).

Ordem de leitura do código: `config.js` → `data.js` → `engine.js` → `app.js`. Todos têm comentários em português. O `README.md` cobre a instalação; este documento cobre o entendimento.